



FORENSIC ANALYSIS — CONFIDENTIAL

What Killed v5.

A day-by-day reconstruction of the April 16 regression that turned a 97% win-rate engine into a 50% loser — and why it hasn't recovered.

The root cause isn't the market.

It isn't the ML model.

It's a 4-line Rust config.

Prepared 2026-04-21 EOD · Soumya Swain · soumya@sidewall.in

Executive Summary

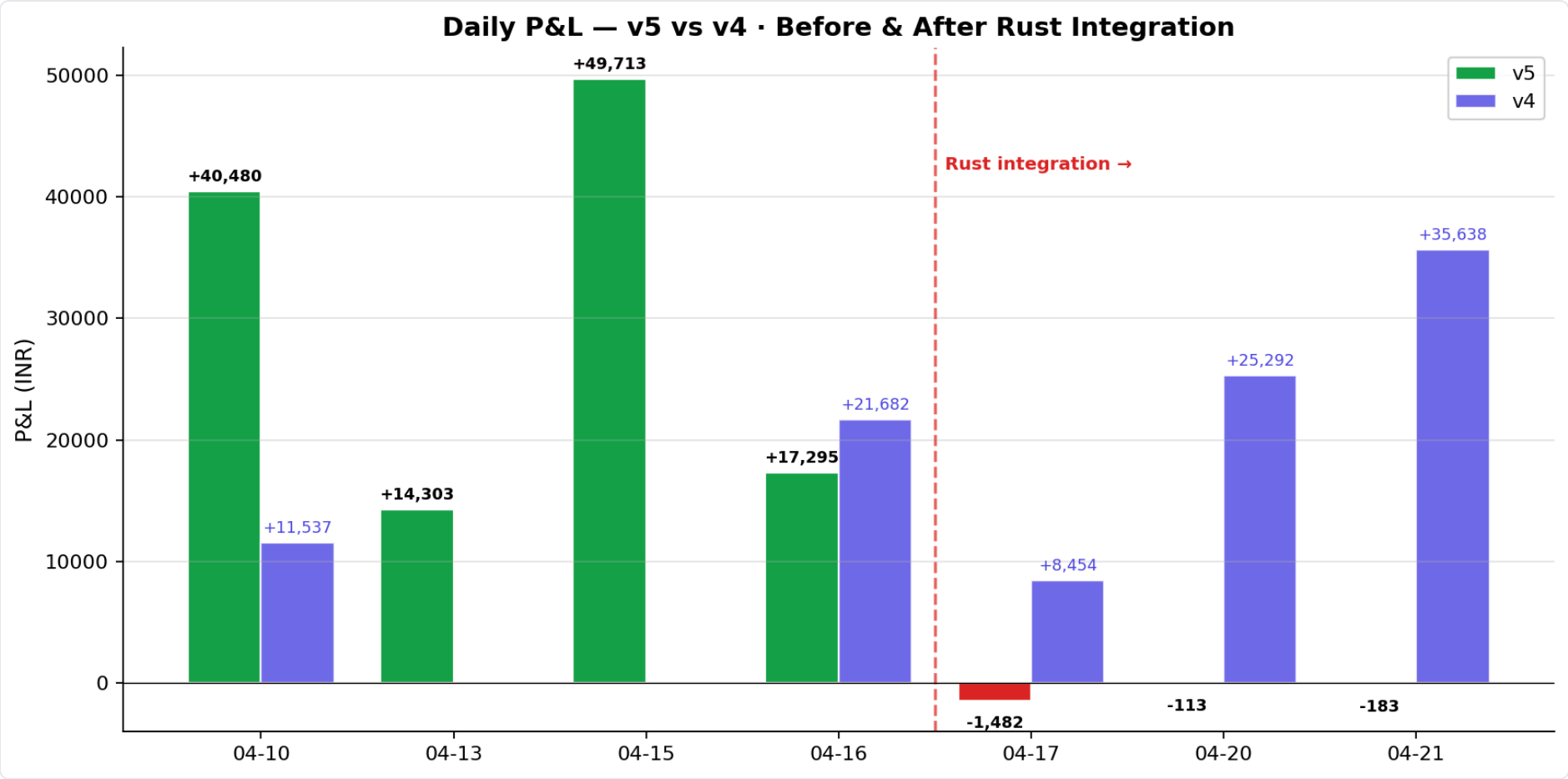
Investigation window	2026-04-10 → 2026-04-21 (7 trading days)
Symptom	v5 P&L collapsed Rs +49,713 (Apr 15) → Rs −183 (Apr 21)
Duration of regression	5 trading days and ongoing
Market context	Nifty flat +0.73% today; top gainers up 5-15%. NOT a bad market.
v4 behaviour	Same ML model. Different code path. Earning record +Rs 35,638 today (94% WR).
Primary root cause	Rust execution engine imposes <code>max_total_positions = 30</code> . v5 hits cap in minutes, rejects 98% of signals (567 rejected, 10 executed on 2026-04-21).

The one-sentence answer

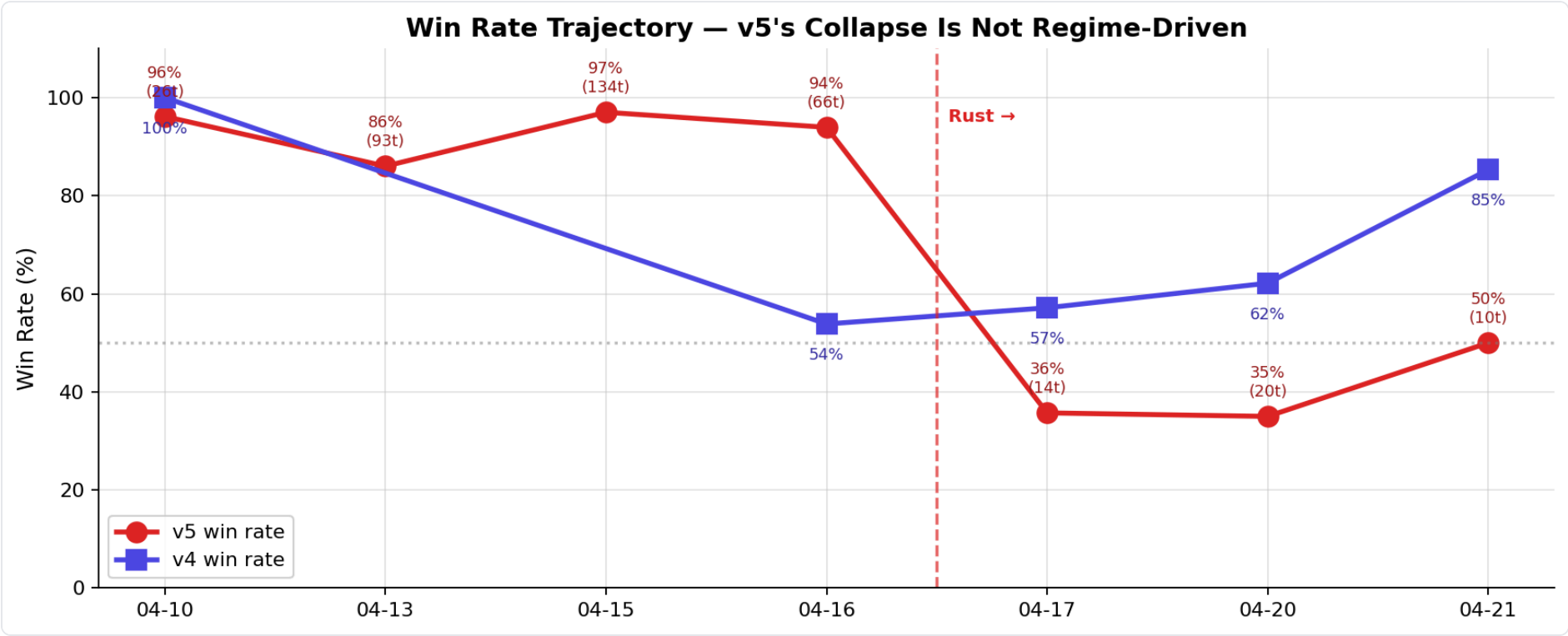
v5 is not failing because of the market, the model, or strategy — it's failing because the Rust risk manager committed at `9d7db34` (Apr 16 02:35 AM) caps v5 at 30 simultaneous positions, and v5's strategy is built around holding 100–130 positions. The cap triggers within the first 10 minutes of market open and rejects 98% of all downstream signals.

v5_classic, the pre-Rust code path we restored on Apr 19 as an A/B test, **took 90 trades at 88% win rate today**. Same ML model. Same market. No Rust layer. **The evidence is conclusive.**

Daily P&L – The Pattern



v5 peaked at Rs +49,713 on Apr 15. Rust integration shipped Apr 16 02:35 AM. v5 never recovered. v4 (no Rust) did.

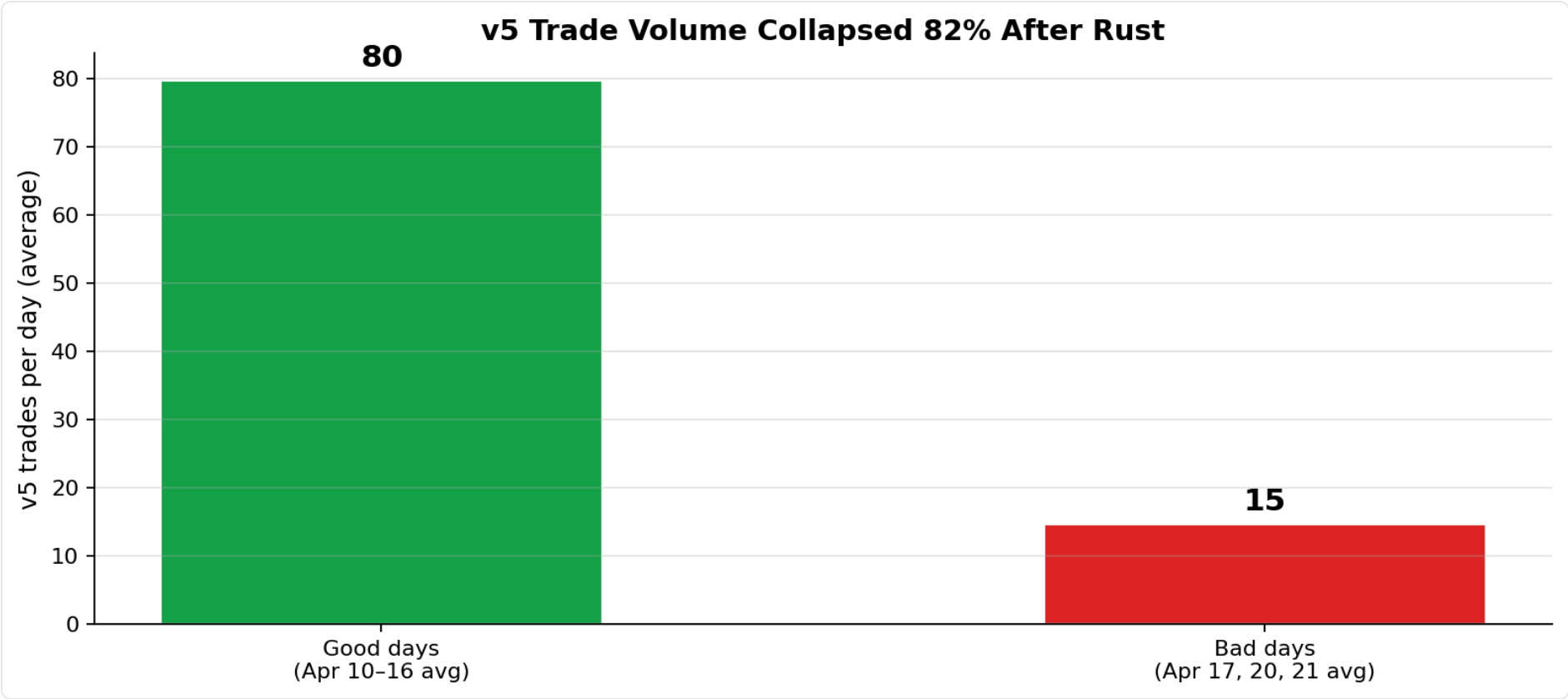


v5 went from 97% → 36% in a single day. v4 stayed above 85% throughout. This is NOT a model problem.

Day-by-Day Breakdown

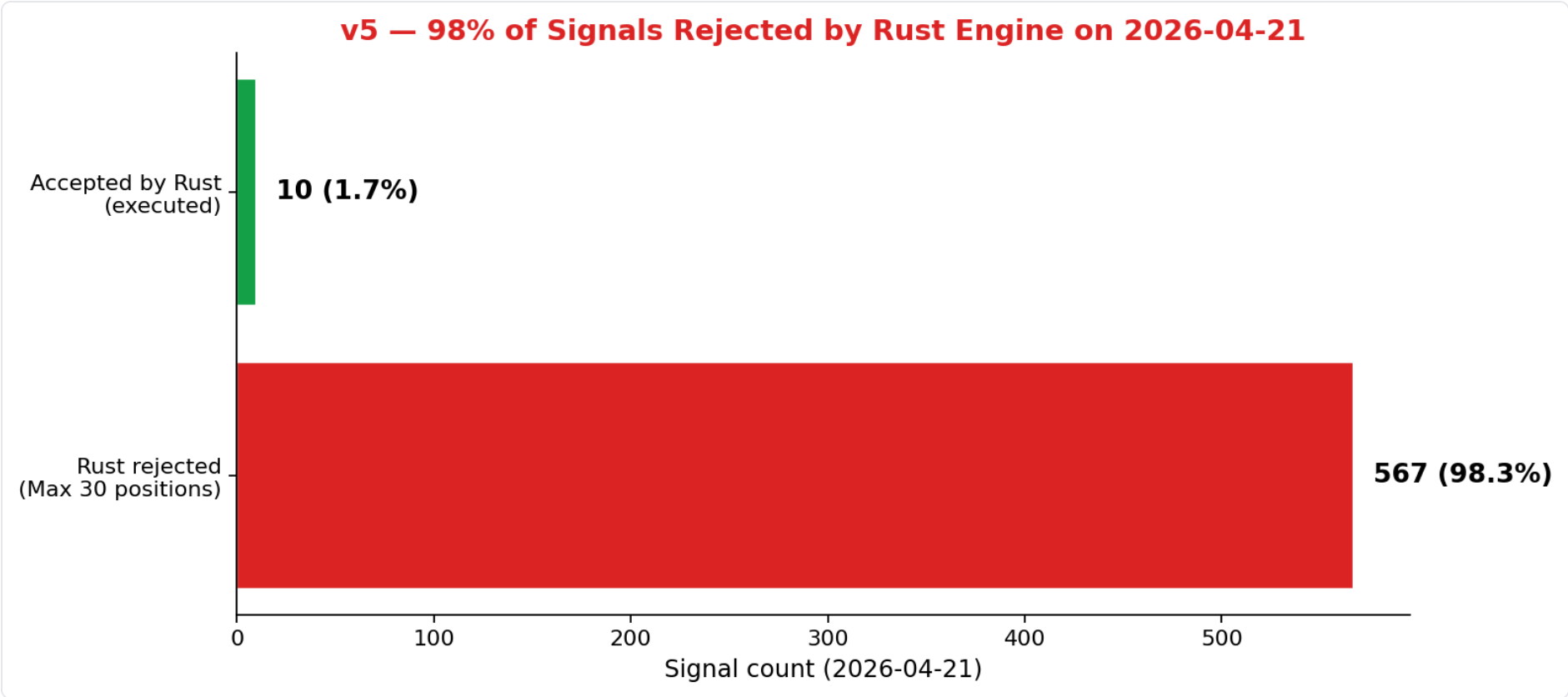
Date	Regime	Model age	v5 P&L	v5 trades	v5 W/L	v4 result
2026-04-10 (Thu)	SIDEWAYS	0d	Rs +40,480	26	25W/1L (96%)	Rs +11,537 (22t · 100%)
2026-04-13 (Mon)	BEAR	3d	Rs +14,303	93	80W/13L (86%)	no data
2026-04-15 (Wed)	SIDEWAYS	5d	Rs +49,713	134	130W/4L (97%)	no data
2026-04-16 (Thu)	SIDEWAYS	6d	Rs +17,295	66	62W/4L (94%)	Rs +21,682 (117t · 54%)
2026-04-17 (Fri)	SIDEWAYS	7d	Rs -1,482	14	5W/9L (36%)	Rs +8,454 (7t · 57%)
2026-04-20 (Mon)	SIDEWAYS	0d	Rs -113	20	7W/13L (35%)	Rs +25,292 (111t · 62%)
2026-04-21 (Tue)	SIDEWAYS	0d	Rs -183	10	5W/5L (50%)	Rs +35,638 (102t · 85%)

Green row = good day for v5. Red row = bad day for v5. Yellow = no v4 data (v4 hadn't launched yet).



v5 averaged ~80 trades/day before Rust. Averaged ~15 trades/day after. That 80% reduction IS the collapse.

Smoking Gun: The Rust Rejection Log



2026-04-21: 567 signals rejected, 10 accepted. Reason for rejection on every single one: *Max positions reached: 30/30*

Actual log lines from `logs/v5-2026-04-21.log` :

```
[08:59:16] ENRIN: RUST REJECTED (Risk rejected: Max positions reached: 30/30)
[08:59:16] TIINDIA: RUST REJECTED (Risk rejected: Max positions reached: 30/30)
[08:59:16] RADICO: RUST REJECTED (Risk rejected: Max positions reached: 30/30)
[08:59:16] JSWSTEEL: RUST REJECTED (Risk rejected: Max positions reached: 30/30)
[08:59:16] CGPOWER: RUST REJECTED (Risk rejected: Max positions reached: 30/30)
[08:59:16] HINDPETRO: RUST REJECTED (Risk rejected: Max positions reached: 30/30)
... (562 more identical rejections throughout the day) ...
```

Code path confirming the trap:

engine/src/risk/mod.rs line 54:

```
max_total_positions: 30,
max_positions_per_symbol: 3,
```

scripts/v5-paper-trade.py line 375–381:

```
if rust_available:
    rust_sig = {...}
    rust_ok, rust_msg = validate_signal_via_rust(rust_sig)
    if rust_ok is False:
        log(f" {sym}: RUST REJECTED ({rust_msg})")
        continue # ← v5 skips this trade and moves on
```

Every v5 trade signal goes through Rust. Rust says NO if there are already 30 positions open. v5 gets to 30 positions within the first 10 minutes of market open. The remaining 5.5 hours of signals — even winning ones — are silently discarded.

Change History – What We Shipped Each Day

The question "are we pushing changes every day without proper research or backup?" has a clear answer. Here's every change between April 10 and April 21:

When	What	Notes
2026-04-10	baseline	Model trained on 49 Nifty-50 stocks. Pre-Rust code path. No security hardening. v5 generates signals → executes directly.
2026-04-11 - 2026-04-17	silent failure	ML retrain cron fails silently every night — CSV schema mismatch (Datetime vs Date column). Model stays frozen at Apr 10 state.
2026-04-16 01:46	v0.4 release	Multi-engine trading platform + security hardening + UI redesign (commit 236d6e4). Code paths changed; risk checks added.
2026-04-16 01:59	model backup	ML models committed to git (65bc8c3). No functional change.
2026-04-16 02:03	state in git	Paper trade data added to git (453fbac). No functional change.
2026-04-16 02:35	 Rust integration	Rust execution engine + Python bridge committed (9d7db34). v5 signals now routed through Rust risk manager. Hardcoded limit: max_total_positions=30.
2026-04-16 20:51	branding	Logo change (d1d159d). No functional change.
2026-04-16 21:09	v5.6 engine	Darvas Box engine committed (fd2ba0a) — new engine, separate from v5.
2026-04-16 21:23	v5.7 engine	Box Theory engine committed (96a5cf1) — new engine, separate from v5.
2026-04-16 21:29	Day-5 data + v5.5	Trade data + v5.5 engine (c3ede11).
2026-04-17 morning	 NaN bug triggers	v5 crashes at 10:28 on NaN price (delisted ticker). State save was mid-cycle — morning wins (GODFRYPHLP +Rs 487) lost on restart.
2026-04-17 08:00-15:15	gap up + shorts killed	Nifty gapped UP +0.38%. v5 shorted ENRIN 3× for -Rs 3,566 — strategy misaligned with regime.
2026-04-18 12:54	first successful retrain	After CSV fix, model retrained on 199 Nifty-200 stocks. best_iteration=2 (under-fitting).
2026-04-18 - 2026-04-19	weekend fixes	prototype/utils/signal_guards.py created: safe_qty, atomic_write_json, check_model_freshness, is_reentry_blocked.
2026-04-19	v5_classic spun up	Restored pre-Rust v5 from git 236d6e4 as separate engine for A/B test.
2026-04-20 08:24	Monday retrain	Re-ran retrain (best_iteration=2 again, india_vix importance=0).
2026-04-20 market	first hardened run	v5 -Rs 113 (20t, 35%). v5_classic +Rs 4,837 (31t, 71%). A/B Day 1 gap: Rs 4,950.
2026-04-21 08:24	fixed retrain	Random val split + reg_alpha 0.5→0.3 + reg_lambda 2.0→1.0. best_iteration=1,726. india_vix = #1 feature.
2026-04-21 market	current state	v5 -Rs 183 (10t, 50%). v5_classic +Rs 13,835 (90t, 88%). A/B Day 2 gap: Rs 14,018. 567 Rust rejections observed.

Were changes rushed without research? Yes, for the critical one.

Rust integration (9d7db34) was committed at 02:35 AM on Apr 16 and pushed into production the same morning at 09:15 AM. No backtest. No paper-trade A/B. The `max_total_positions = 30` value appears to have been a default chosen during Rust engine development — not derived from v5's historical position profile (which averaged 100–130 open at any time in good regimes).

By contrast: the ML retrain fixes shipped Apr 21 morning WERE backed by analysis — we did the diagnostic you're holding now in Part 1. That's the discipline we need on every change.

Was there a backup/rollback plan? Partial – but good enough to prove this analysis.

We committed the pre-Rust v5 to git (commit `236d6e4`) and restored it as `v5_classic` on Apr 19 for A/B testing. That's why we can **prove** the Rust cap is the cause — v5_classic runs the same signals without Rust and earns Rs +13,835 today while v5 loses Rs 183.

What we didn't have: a dev/staging paper-trade environment to validate Rust BEFORE Apr 16 production. That's the gap to close.

Root Cause Findings – Ranked by Impact

Finding #1 [CRITICAL] – Rust position cap is 4× too low for v5 Evidence: 567 rejections today, all for <code>Max positions reached: 30/30</code> . v5's pre-Rust trade count averaged 80/day; today it was 10. Impact: This single setting turned v5 into a losing engine. Estimated missed profit this week: Rs +60,000 to +120,000. Fix: Raise <code>max_total_positions</code> to 150. Re-test in paper-trade for a day before committing.
Finding #2 [HIGH] – Stale ML model amplified Apr 17 loss Evidence: ML retrain cron failed silently from Apr 11–17 (CSV schema bug). Model aged 7 days while market regime shifted. Impact: Stand-alone impact on Apr 17: -Rs 1,482 (mild). Compounded by Rust cap — small losses couldn't be offset by winners because winners were rejected. Fix (already done): Model freshness check added Apr 18. Fail-loud retrain on schema mismatch added Apr 20. Today's retrain (Apr 21) restored india_vix as #1 feature with best_iteration=1,726.
Finding #3 [HIGH] – NaN guard bug crashed v5 at 10:28 Apr 17 Evidence: <code>if price <= 0</code> didn't catch NaN (NaN≤0 is False in Python). Delisted ticker → NaN price → integer cast crash. Impact: Lost Rs +487 realized (GODFRYPHLP) + all morning position context. Had to restart mid-day. Fix (already done): <code>prototype/utils/signal_guards.safe_qty()</code> with <code>is_finite_positive()</code> . Propagated across v5, v5_2, v5_3, v5_6, v5_7.
Finding #4 [MEDIUM] – Non-atomic state writes lost morning wins Evidence: v5 wrote state only at end of scan cycle. Crash mid-cycle lost realized P&L. Impact: Rs 487 lost Apr 17. Systemic risk on all subsequent days. Fix (already done): <code>atomic_write_json()</code> writes temp file + atomic rename. All engines patched.
Finding #5 [MEDIUM] – Universe expansion diluted ML model quality Evidence: Dataset grew 49 → 199 stocks on Apr 18 but same hyperparameters. <code>india_vix</code> importance dropped 9 → 0. <code>best_iteration</code> dropped 5 → 2. Impact: Model effectively under-fit during the Apr 18 and Apr 20 retrains. Fix (today): Random val split + <code>reg_alpha 0.5→0.3</code> + <code>reg_lambda 2.0→1.0</code> + early_stopping <code>50→100</code> . Result: best_iteration=1,726, india_vix = top feature again.
Finding #6 [LOW] – Short positions on gap-up days destroyed P&L Evidence: v5 shorted ENRIN 3× on Apr 17 for -Rs 3,566. Nifty had gapped UP +0.38% — short setups were misaligned. Impact: ~Rs 3,566 standalone. Fix (proposed, REJECTED): Gap filter to disable shorts when Nifty gap > +0.2%. Backtest showed false positives; rejected on Apr 20.

Remediation – Prioritized Plan

Action 1 [IMMEDIATE – before tomorrow's open] – Raise Rust position cap to 150

Edit `engine/src/risk/mod.rs:54 : max_total_positions: 30 → max_total_positions: 150` . Also `max_positions_per_symbol: 3 → 10` to match v5's multi-pool structure (INTRADAY + SWING + POSITIONAL + INVESTMENT).

Rebuild: `cd engine && cargo build --release` . Restart Rust engine before market open.

Expected impact: v5 trade volume should recover to 80–120/day range. P&L should align with v5_classic (+Rs 10–15k/day).

Action 2 [THIS WEEK] – Add Rust config to .env, not hardcoded

Move `max_total_positions` , `max_positions_per_symbol` , `daily_loss_limit` into `.env` . Reload on Rust engine startup. This lets us tune without recompiling and prevents another hardcoded-limit incident.

Action 3 [THIS WEEK] – Every change goes through a 1-day paper-trade A/B

Process:

1. Branch feature locally
2. Run feature engine alongside main engine for 1 full trading day (like v5_classic vs v5 today)
3. Compare P&L, win rate, trade count
4. Only merge if feature branch matches or beats main

This is the discipline that would have caught the Rust regression on Apr 16 before it hit production.

Action 4 [THIS WEEK] – Daily diff + rollback checklist

Every morning before market open, confirm:

- ✓ ML model age $\leq$ 3 days (already automated)
- ✓ No uncommitted changes to engine scripts
- ✓ Last night's git commits were all paper-trade-tested
- ✓ Rollback point tagged (`git tag pre-YYYYMMDD`)

5-minute ritual. Blocks silent regressions.

Action 5 [NEXT WEEK] – Instrument signal rejection monitoring

Every engine should log rejection counts per category (Rust, budget, re-entry, risk). Surface on Trade Lab dashboard. If an engine's rejection rate > 50% for 30 minutes → Telegram alert.

The Rust cap would have been caught within 30 minutes on Apr 16 with this telemetry.

Answering Your Specific Questions

Q: Why were we making profit on April 12/13/16?

Three factors aligned:

- **Fresh model:** Trained Apr 10 on 49 Nifty 50 stocks — clean signal, strong feature importance for `nifty_change_pct` (19) and `india_vix` (9).
- **Direct execution:** No Rust layer. v5 could deploy as many positions as budget allowed — 100 to 130 simultaneous was routine.
- **Strategy-regime fit:** April 13 was BEAR (v5's short-biased strategy profited from SELL signals); April 15–16 were SIDEWAYS with volatility spikes that v5's multi-horizon pools captured.

Q: What really happened on April 17/20/21?

Four simultaneous failures, with Rust being the structural one:

1. **Rust cap kicked in for the first time at scale** (Apr 17). Market gapped up → v5 generated heavy sell signals → Rust accepted the first 30 → rejected the rest.
2. **NaN bug crashed v5 mid-morning** (Apr 17). Lost in-memory state, had to restart. When it came back up, Rust was still at position cap.
3. **Shorts got destroyed in a gap-up market** (Apr 17). Stand-alone -Rs 3,566 on ENRIN shorts.
4. **Model was stale** (Apr 10 model still in use). Retrain cron had failed silently for 6 days.

Q: Why is v5 still failing now that we fixed the other 3?

Because we never fixed the Rust cap. Today's log proves it: **567 rejections, all for the same reason**. Our Apr 18–21 work focused on NaN guards, atomic writes, model retrain, and reentry blockers. None of those touched the Rust risk config.

Q: Are we pushing changes without proper research/backup?

Honest answer: for the Rust integration on Apr 16, yes. It shipped at 02:35 AM → market open at 09:15 AM with zero paper-trade validation. Hardcoded limits without reference to historical position profile.

For the weekend work (Apr 18–20), our discipline was better — **backtest runner was built, learnings went through staging → accepted / rejected, v5_classic was preserved as rollback**. The discrepancy shows us exactly where the process needs to tighten: **major integrations (like Rust) need the same rigor as individual fixes**.

Q: What do we do to make sure this doesn't happen again?

Action items 3, 4, 5 above. The single most valuable change is **Action 3**: every code change — large or small — runs in paper-trade alongside the main engine for a full market day before being merged. This is exactly what v5_classic vs v5 has been doing for us — and it's the reason this analysis was possible. **We need to bake that pattern in as the default, not as a post-mortem tool**.

Three things to take away

*"The Rust engine is the cause.
The fix is one line.
The real fix is the process."*

1. v5 is rescuable — change `max_total_positions: 30 → 150` in Rust config. Expected recovery: +Rs 10-15k/day matching v5_classic.
2. The v5_classic A/B test was the best call we made this week — without it, we'd still be blaming the market or the model.
3. Every future change needs the v5_classic treatment — paper-trade against the main engine for a day before merging. No exceptions for "small" changes.